Computer Organization
Laboratory Exercise 1

Using a Nios® V System

This is an introductory exercise for the Altera® Nios® V processor. The exercise uses a pre-defined computer system called the *DE1-SoC Computer* that includes Nios V and various peripheral devices. Documentation for this computer system is available in the document *DE1-SoC Computer System with Nios V*, which is available under `Computer Systems` in the `Computer Organization` course on the FPGAcademy.org website. The computer system is implemented as a circuit that is downloaded into the FPGA device on a DE1-SoC board. Details about the features of this board can be found in the `Teaching and Project Boards` section of FPGAcademy.org. This exercise illustrates how programs written in the Nios V assembly language can be executed on the DE1-SoC board.

In this introductory exercise you will begin learning how to develop programs written in the Nios V assembly language, which can be executed in the *DE1-SoC Computer*. You will need to be familiar with the Nios V processor architecture and its assembly language. An overview of Nios V can be found in the tutorial *Introduction to the Nios V Processor*, which is available as part of the `Computer Organization System Design` tutorials on FPGAcademy.org.

To develop and "execute" programs for Nios V you will use two different software tools, described below: the *CPUlator* and the *Gnu Project Debugger* (*GDB*).

CPUlator: The *CPUlator* is a web-based simulation tool. It uses software techniques to *simulate* the functional behavior of the components in the *DE1-SoC Computer*, including the Nios V processor, memory, and a number of I/O devices. The CPUlator is an excellent tool for developing and debugging Nios V programs—it runs inside a web browser, is easy to use, and does not require any hardware. The CPUlator is introduced in Part I, below.

GDB: The *Gnu Project Debugger* is a set of industry-standard software development tools. It allows you to develop and debug Nios V programs that *execute in hardware* on a DE1-SoC board. The GDB tools are introduced in Part VI.

Part I

In this part of the lab exercise you are to watch a short video that provides an introduction to the *CPUlator* tool. As you will see in the video, the *CPUlator* is a full-featured software development environment that allows you to compile (assemble) and debug both assembly and C code for Nios V.

Use the following `URL` to access the video:

https://youtu.be/F4PDYijJX8U

Part II

Now, we will explore some features of the *CPUlator* by working with a simple Nios V assembly language program. Consider the program given in Figure 1, which finds the largest number in a list of 32-bit integers that is stored in the memory.

Note that some sample data is included in this program. The word (4 bytes) at the label *result* is reserved for storing the result, which will be the largest number found. The next word, N, specifies the number of entries in the list. The words that follow give the actual numbers in the list.

```
# Program that finds the largest number in a list of integers
.global _start
_start:     la      t0, result      # t0 = pointer to the result
            lw      t1, 4(t0)       # t1 = counter, initialized with N
            addi    t2, t0, 8       # t2 = pointer to the first number
            lw      t3, (t2)        # t3 = largest found so far
loop:       addi    t1, t1, -1      # decrement counter
            beqz    t1, done        # done when counter is 0
            addi    t2, t2, 4       # point to the next number
            lw      t4, (t2)        # get the next number
            bge     t3, t4, loop    # compare to largest found
            mv      t3, t4          # remember new largest
            j       loop

done:       sw      t3, (t0)        # store result
stop:       j       stop            # wait here

result:     .word   0               # result will be stored here
N:          .word   7               # number of entries in the list
numbers:    .word   4, 5, 3, 6      # numbers in the list
            .word   1, 8, 2         # ...
```

Figure 1: Assembly-language program that finds the largest number.

Make sure that you understand the program in Figure 1 and the meaning of each of its instructions. Note the extensive use of comments in the program. You should always include meaningful comments in programs that you will write!

Perform the following:

1. Create an assembly-language source-code file for the program in Figure 1, and name the file *part2.s* (this file is included in the *design files* for this lab exercise). Then, open the *CPUlator* web-site and set its system parameters to choose the `RISC-V RV32` processor and `DE1-SoC` board (the corresponding `URL` for the *CPUlator* web page should be https://cpulator.01xz.net/?sys=rv32-de1soc. As indicated in Figure 2, click on the `File` command near the top of the *CPUlator* window and then select `Open...`. Next, in the dialogue depicted in Figure 3, browse in your computer's file-system to choose the *part2.s* file and then click `Open`.

   The assembly program should be displayed in the `Editor` pane of the *CPUlator* window, as illustrated in Figure 4. You can make changes to your code in this `Editor` pane if needed by simply selecting text with your mouse and making edits using your keyboard.

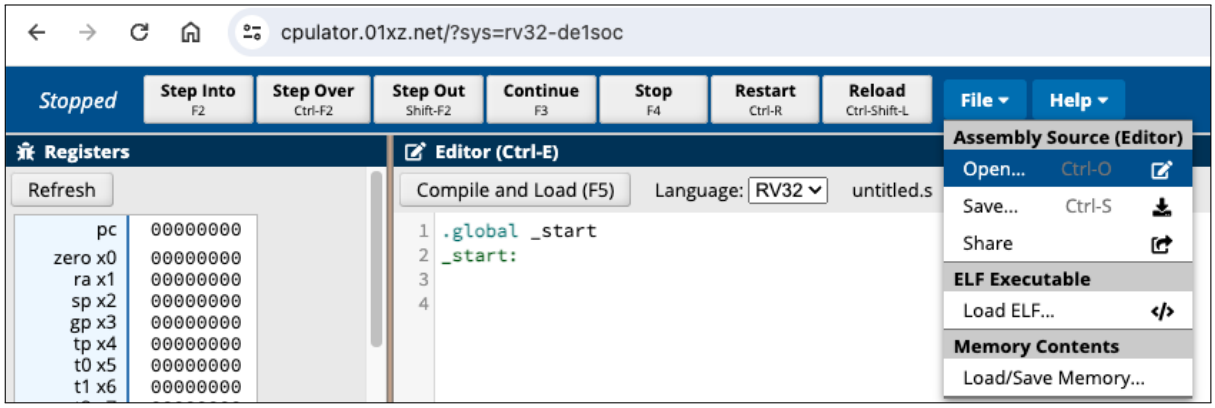

Figure 2: The `File` menu in *CPUlator*.

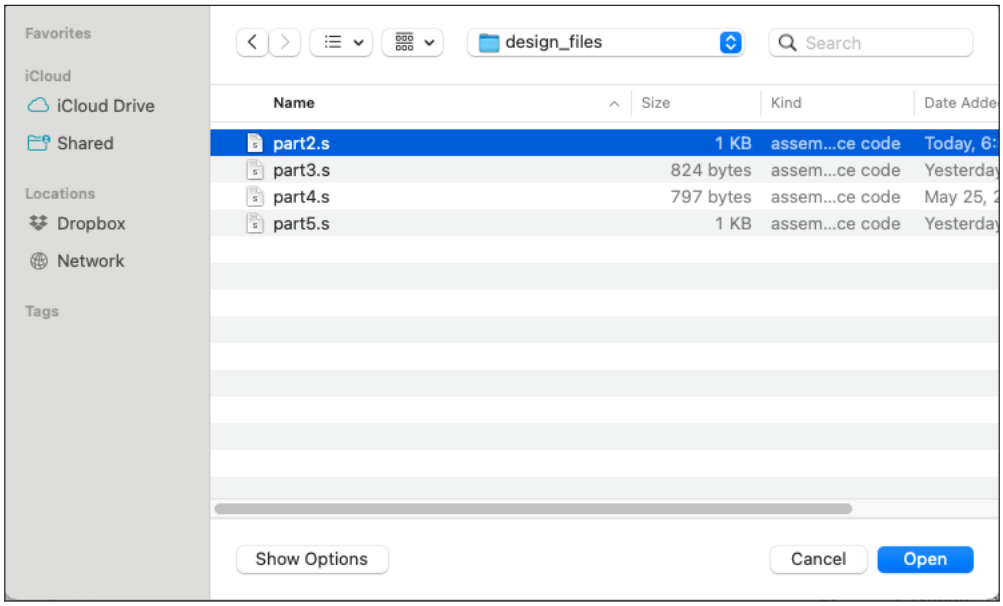

Figure 3: Selecting the *part2.s* file.

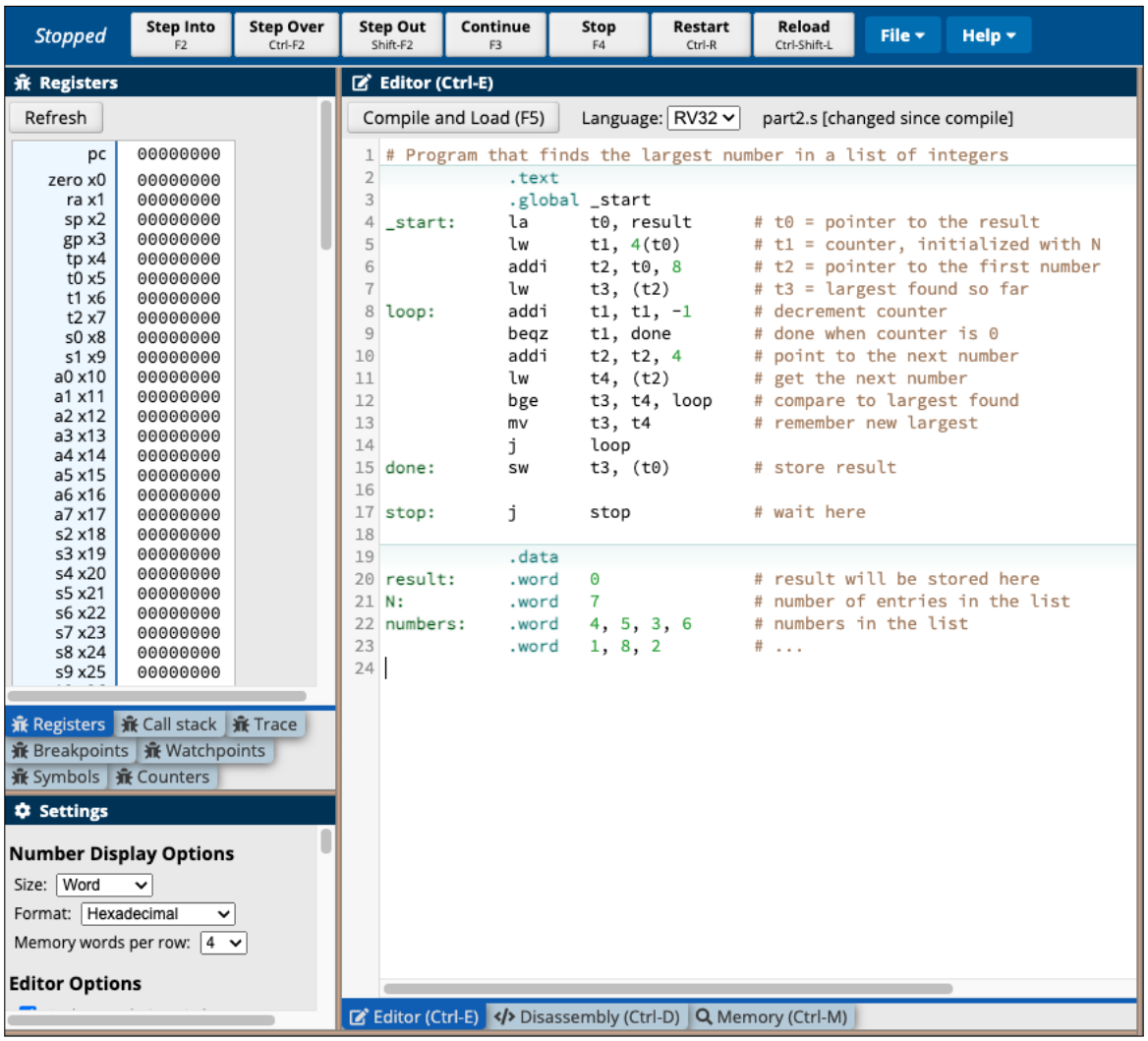

Figure 4: The assembly-language program in the *CPUlator* `Editor` pane.

2. As indicated in Figure 5, click on the `Compile and Load` command to *assemble* your program and *load* it into the *memory* that is part of the computer system being simulated within the *CPUlator* tool. You should see the message displayed in Figure 6, in the `Messages` pane, which reports a successful compilation result. If not, then you may have inadvertently introduced an error in the program code; fix any such errors and recompile.

   Once the compilation is successful, the *CPUlator* window automatically displays the `Disassembly` pane, shown in Figure 7. This pane lets you see the machine code for the program and gives the address in the *memory* of each machine-code word. The `Disassembly` pane shows each instruction in the program twice: once using the original source code and a second time using the actual instruction found by *disassembling* the machine code. This is done because the *implementation* of an instruction may differ, in some cases, from the *specification* of that instruction in the source code (examples where such differences happen will be shown in class). Note: you can change the way that code is displayed in the CPUlator by using its `Settings` menu, which is on the left hand side of the CPUlator window (for example, changing `Disassembly Options` from *Some* to *None* will ensure that each instruction is displayed only once).

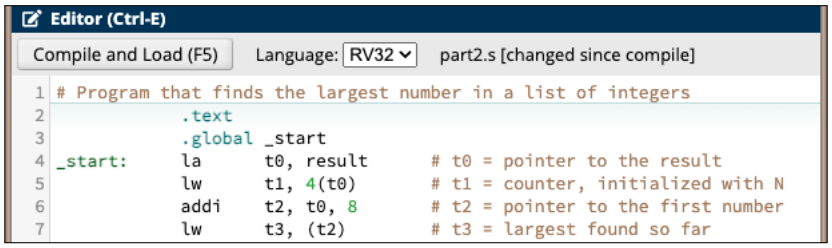

Figure 5: Compiling and loading the program.

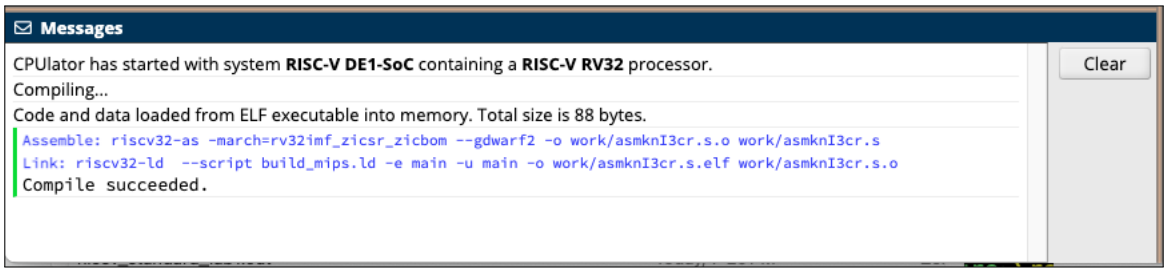

Figure 6: The `Messages` pane.

3. Select the `Continue` command near the top of the *CPUlator* window. This command "executes" the program on the Nios V processor that is part of the computer system being simulated within the *CPUlator* tool. As illustrated in Figure 7, the program runs to the line of code labeled `stop`, at memory address `0x30`, where it remains in an endless loop. Select the `Stop` command to halt the program's execution. Note that the largest number found in the sample list is 8 as indicated by the contents of register `t3`. This result is also stored in memory at the label *result*.

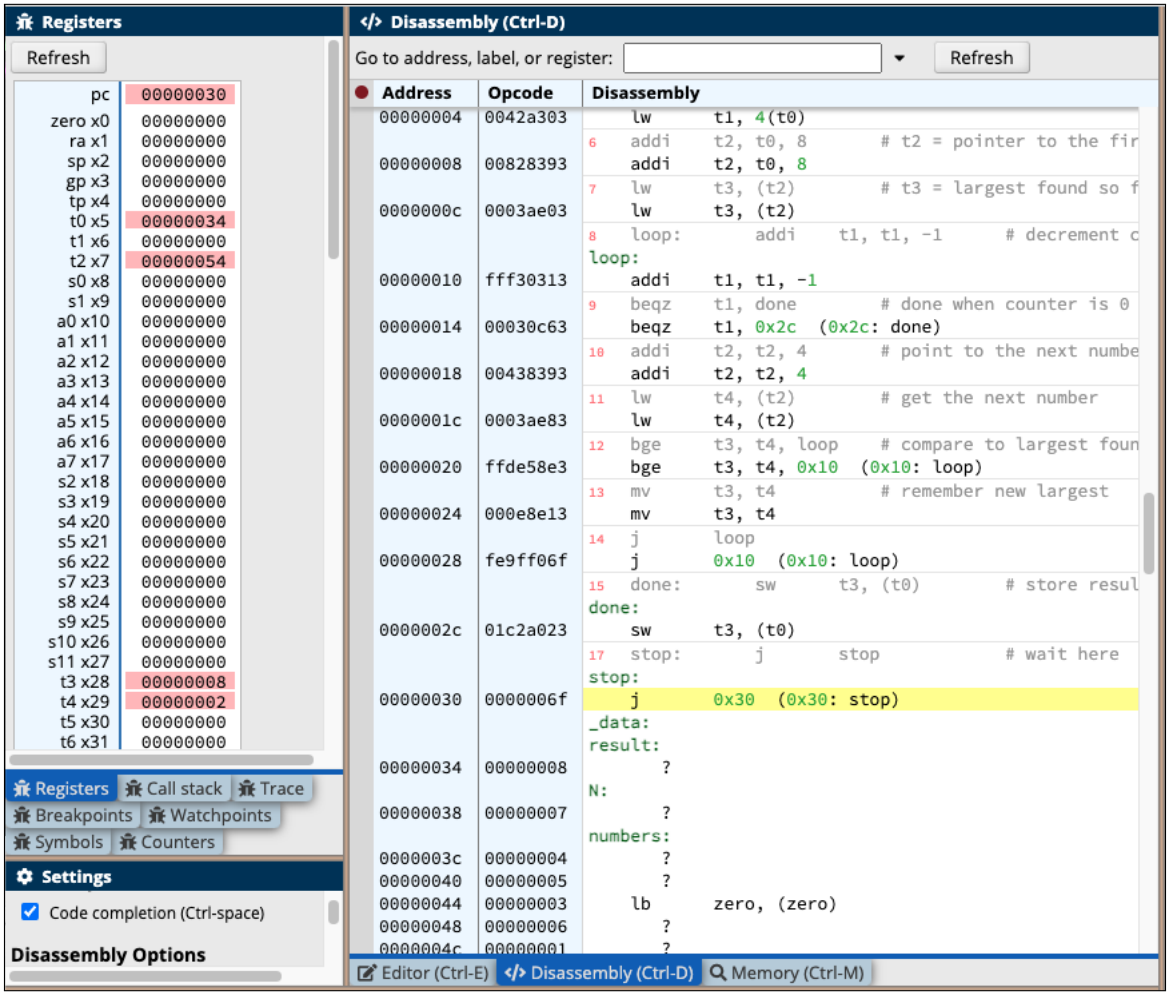

Figure 7: The result of executing the program.

The address of the label *result* is `0x00000034`, which can be seen near the bottom of Figure 7. Also, you may notice that the `Disassembly` pane attempts to figure out the machine code at this location, assuming that it represents a processor instruction; it does not, and so the resulting instruction displayed (`?`) is not meaningful.

Use the *CPUlator's* `Memory` pane, as illustrated in Figure 8, to verify that the resulting value 8 is stored in the correct location.

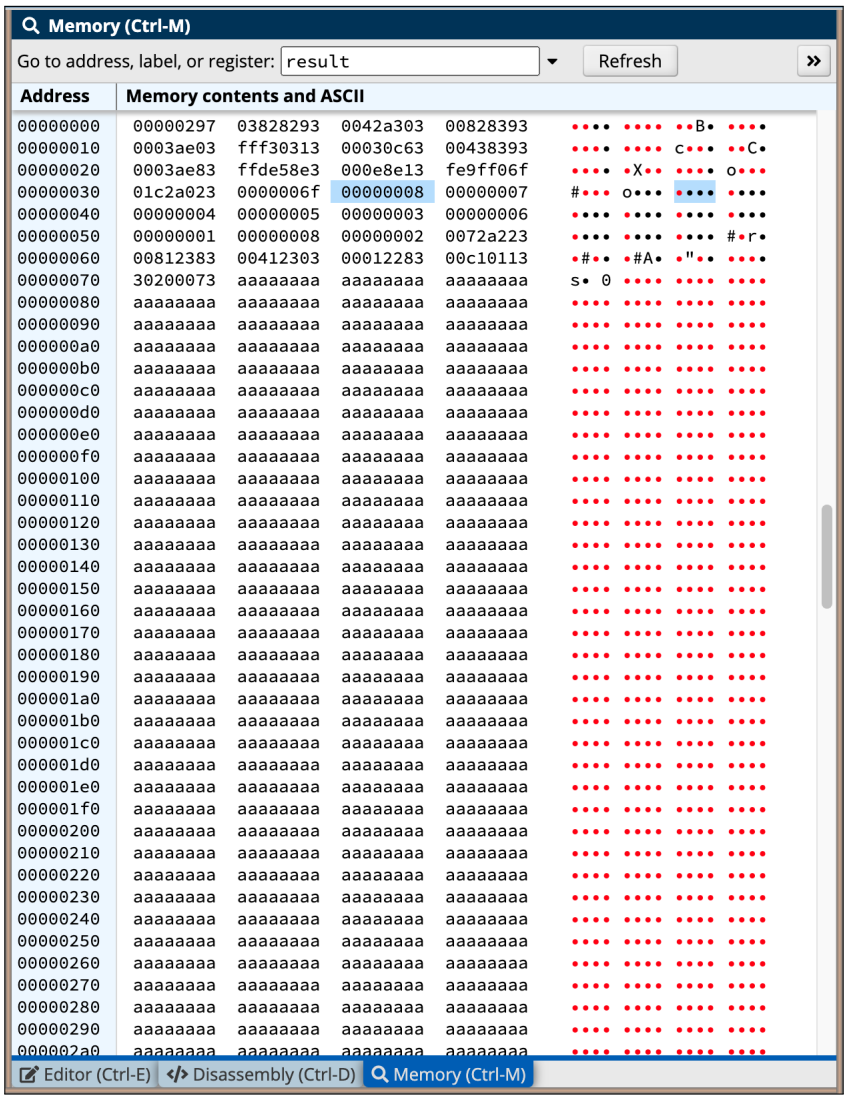

Figure 8: The `Memory` pane.

4. You can return control of the program to the start by clicking on the `Restart` command in *CPUlator*. Do this and then *single-step* through the program by (repeatedly) selecting the `Step Into` command while observing the `Disassembly` pane. Observe how each instruction that is executed affects the contents of the Nios V registers.

5. Double-click on the `pc` register in the *CPUlator* and then change the value of the program counter to 0. This action has the same effect as selecting the `Restart` command.

6. Now set a breakpoint at address `0x00000028` by clicking on the gray bar to the left of this address, as illustrated in Figure 9. Select the `Continue` command to run the program again and observe the contents of register `t3` when the instruction at the breakpoint, which is `j loop`, is reached. Use `Continue` to repeatedly run the program, which will stop at the breakpoint in each iteration of the loop, and monitor the contents of register `t3` as the program searches for the largest number in the list.

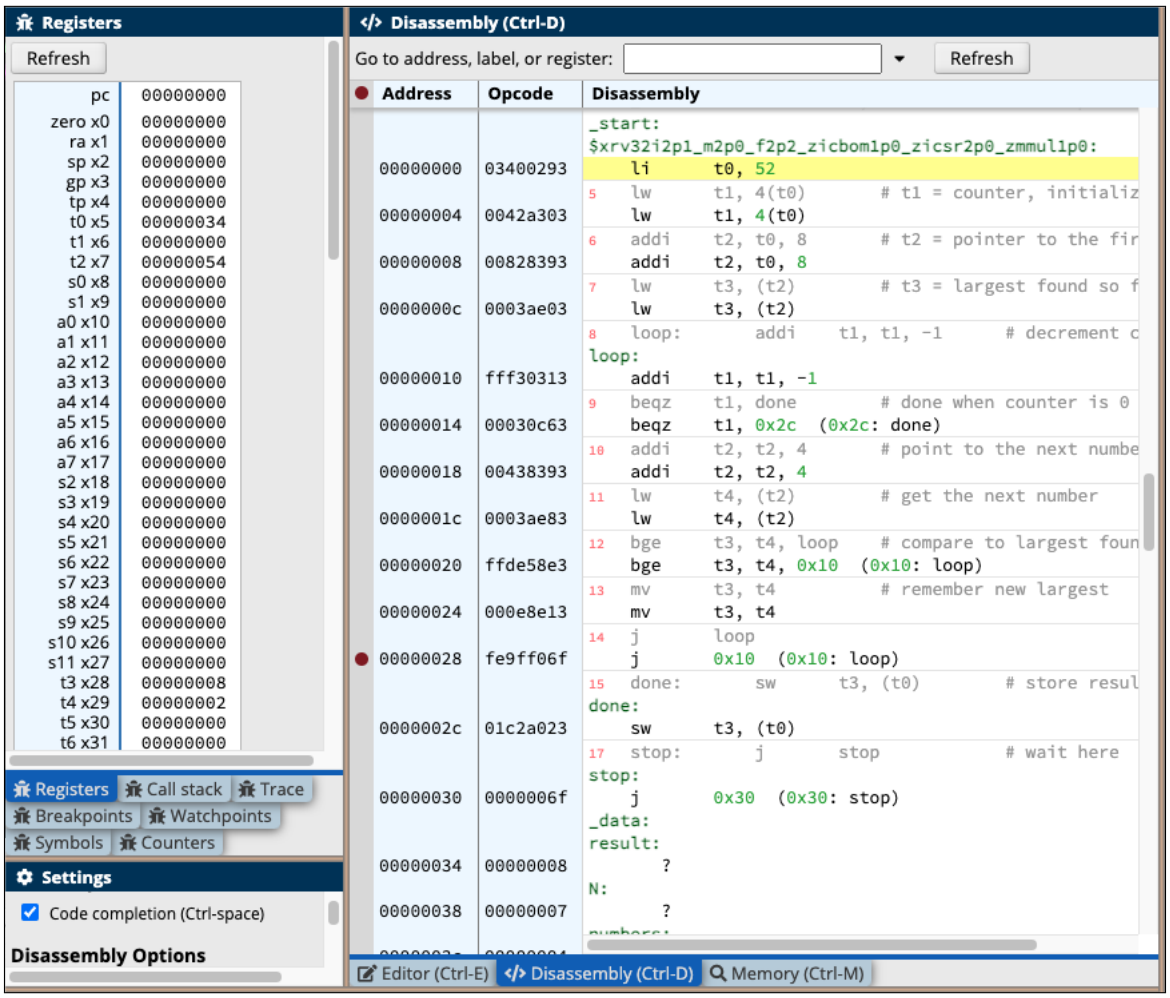

Figure 9: Setting a breakpoint.

Part III

Implement the task in Part II by modifying the program in Figure 1 so that it uses a subroutine. The subroutine, named *large*, has to find the largest number in a list. The main program passes the number of entries and the address of the start of the list as parameters to the subroutine via registers *a0* and *a1* respectively. The subroutine returns the value of the largest number to the calling program via register *a0*. A suitable main program is given in Figure 10.

Use the *CPUlator* tool to assemble, load, execute, and debug (as needed!) your program.

```
# Program that finds the largest number in a list of integers
.global _start
_start:     la      s0, result
            lw      a0, 4(s0)     # a0 holds number of list elements
            addi    a1, s0, 8     # a1 points to the list
            jal     ra, large
            sw      a0, (s0)      # store result

stop:       b       stop          # wait here

# Subroutine finds largest number in list of a0 elements starting at
# address in a1. Result is returned in a0
large:      ...
            .
            .
            ret

result:     .word   0             # result will be stored here
N:          .word   7             # number of entries in the list
numbers:    .word   4, 5, 3, 6    # numbers in the list
            .word   1, 8, 2       # ...
```

Figure 10: Main program for Part III.

Part IV

The program shown in Figure 11 reads a string of characters (bytes) from the memory and reversing the string in-place. The string is created in the memory by using the `.asciz` assembler directive, which generates the ASCII code for each character in the string and places these bytes into the memory starting at the address of the *string* label.

The program works by first setting a pointer to the beginning of the string, and then identifying, using the loop labeled `zloop`, the location in memory of the last character in the string. Next, in the loop labeled `reverse`, the program swaps the two characters at the ends the string, and then the ones next to those on each end, and so on, working toward the middle of the string until done.

Perform the following:

1. Create an assembly-language source-code file for the program in Figure 11, and name the file *part4.s* (this file is included in the *design files* for this lab exercise). Then, enter this code into the *CPUlator* tool.

2. Make use of the *CPUlator* features, such as single-stepping, setting breakpoints, examining registers and the contents of memory, and so on, to ensure that you thoroughly understand how the code in Figure 11 works. Make sure that you examine the *string* in the `Memory` tab of the *CPUlator*, both before and after it has been reversed. Note that the ASCII characters of the string are displayed on the right-hand side of the `Memory` tab.

```
# Program that reverses a text string

.global _start
_start:     la      s0, string   # s0 pointer to first character
            mv      s1, s0
zloop:      lb      t0, (s1)     # read the character
            beqz    t0, cont     # end of string?
            addi    s1, s1, 1    # point to next character
            j       zloop

cont:       addi    s1, s1, -1   # s1 = pointer to last character
reverse:    bge     s0, s1, stop
            lb      t0, (s0)     # swap characters
            lb      t1, (s1)
            sb      t0, (s1)
            sb      t1, (s0)

            addi    s0, s0, 1
            addi    s1, s1, -1
            j       reverse

stop:       j       stop         # wait here

string:     .asciz  "gnirts a si siht"
```

Figure 11: A program that reverses a text string.

Part V

Implement the task in Part IV by modifying the program in Figure 11 so that it uses two subroutines. One subroutine, named *slen*, has to find the number of characters in the string that is to be reversed (not counting the zero at the end). The main program passes to *slen* a pointer to the first character in the string, in register *a0*. The other subroutine, named *srev*, performs the string reversal. Its parameters are a pointer to the first character in the string, in register *a0*, and a pointer to the last character in the string, in register *a1*.

A suitable main program is given in Figure 12. It reserves two strings by using the *slen* and *srev* subroutines. Use the *CPUlator* to assemble, load, execute, and debug (as needed!) your program. Note that the `.align` assembler directive used in Figure 12 causes each string to be placed in the memory at a 16-byte boundary; this alignment starts each string on a separate line of the `Memory` tab in the *CPUlator*, making it easier for you to visually examine the strings.

```
# Program that reverses text strings

.global _start
_start:     la      s0, string0  # pointer to string
            mv      a0, s0       # pass pointer to subroutine
            jal     slen         # returns string length in a0
            add     a1, s0, a0   # a1 points to string end
            mv      a0, s0       # a0 points to string start
            jal     srev         # reverse the string

            la      s0, string1  # pointer to string
            mv      a0, s0       # pass pointer to subroutine
            jal     slen         # returns string length in a0
            add     a1, s0, a0   # a1 points to string end
            mv      a0, s0       # a0 points to string start
            jal     srev         # reverse the string

stop:       j       stop

# Subroutine that returns the length of a string
# parameter: a0 points to the string
# returns: in a0 the number of characters in the string
slen:       ...
            .
            .
            ret

# Subroutine that reverses a string
# parameters: a0 = string start, a1 = string end
srev:       ...
            .
            .
            ret

.align 4
string0:    .asciz  "gnirts a si siht"
.align 4
string1:    .asciz  "esrever dna daer"
```

Figure 12: Main program for Part V.

Part VI: Introduction to the GDB Tools

This part of the exercise introduces the *Gnu Project Debugger* (GDB) tools, which allows you to develop and debug Nios V code that is being executed on a *real* DE1-SoC board. As stated earlier in this exercise, you cannot use the *CPUlator* to control a real hardware board, because the *CPUlator* only *simulates* the features of the DE1-SoC Computer and does not utilize the actual hardware.

To use GDB with Nios V your computer must be connected by a USB cable to a DE1-SoC board (the cable has to be plugged into the *USB Blaster* port on the board). In a similar manner as described earlier in this exercise (for using the *CPUlator*), you will develop software code that runs on the DE1-SoC Computer, which includes Nios V, memory, and various I/O devices. However, in this case you are not using a *simulation* of the DE1-SoC Computer. Instead, the *DE1-SoC Computer* is implemented as a circuit which is downloaded by the GDB application into the FPGA device on the DE1-SoC board. GDB allows you to control the Nios V processor on the board.

The remaining parts of this exercise require the use of a DE1-SoC board. The GDB application cannot be used without a hardware board attached to your computer.

Perform the following:

1. Before using GDB with Nios V, some necessary software and hardware tools have to be installed onto your computer. The procedure for installing these tools is described in the tutorial *Using GDB with Nios V*, which is available as part of the `Computer Organization System Design` tutorials (version 24.1) on FPGAcademy.org. In the discussion below we assume that you have already installed the required tools onto your computer.

2. Work through the instructions in the tutorial *Using GDB with Nios V* up to the end of Section 4.2. That section of the document introduces key features of the GDB tools, using the assembly code from Part II of this exercise as an example. In the tutorial, the program from Part II is called *largest.s*. You will learn how to assemble and link the assembly code to create an executable file, how to start GDB with this file loaded into the memory on the DE1-SoC board, and how to control the Nios V processor to examine/execute/debug the assembly code and data used in the program.

3. Once you have learned the basic GDB commands from the above example, you may want to gain additional experience with GDB by implementing the Nios V programs from Parts III to V on the DE1-SoC board. As described in the document *Using GDB with Nios V*, the GDB tools are executed by using the *Gnu make* program in the *Windows PowerShell* command-line environment. Thus, you will need to make a new *Makefile* for use with each of the programs from Parts III to V. This *Makefile* will be identical to one used for the *largest.s* example, except you will replace the filename *largest.s* that is specified at the beginning of the *Makefile* with one of *part3.s*, *part4.s*, or *part5.s*.

Copyright © FPGAcademy.org. All rights reserved. FPGAcademy and the FPGAcademy logo are trademarks of FPGAcademy.org. This document is provided "as is", without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose and noninfringement. In no event shall the authors or copyright holders be liable for any claim, damages or other liability, whether in an action of contract, tort or otherwise, arising from, out of or in connection with the document or the use or other dealings in the document.

*Other names and brands may be claimed as the property of others.